Chapter 13 Abstract Classes and Interfaces



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes

- In inheritance, classes become more specific and concrete *with each new subclass*.
  - GeometricObject
  - Rectangle, Circle

- If you move from a subclass back up to a superclass, the classes become more general and less specific.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes

- Class design should ensure that a superclass contains common features of its subclasses.

- Sometimes a superclass is so abstract that it cannot be used to create any specific instances.
  - Creating a GeometricObject ?

- Such a class is referred to as an *abstract class*.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes

- Both **Circle** and **Rectangle** contain the **getArea()** and **getPerimeter()** methods for computing the area and perimeter of a circle and a rectangle.

- Since you can compute areas and perimeters for all geometric objects, it is better to define the **getArea()** and **getPerimeter()** methods in the **GeometricObject** class.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes

- However, these methods cannot be implemented in the **GeometricObject** class, because their implementation depends on the specific type of geometric object.

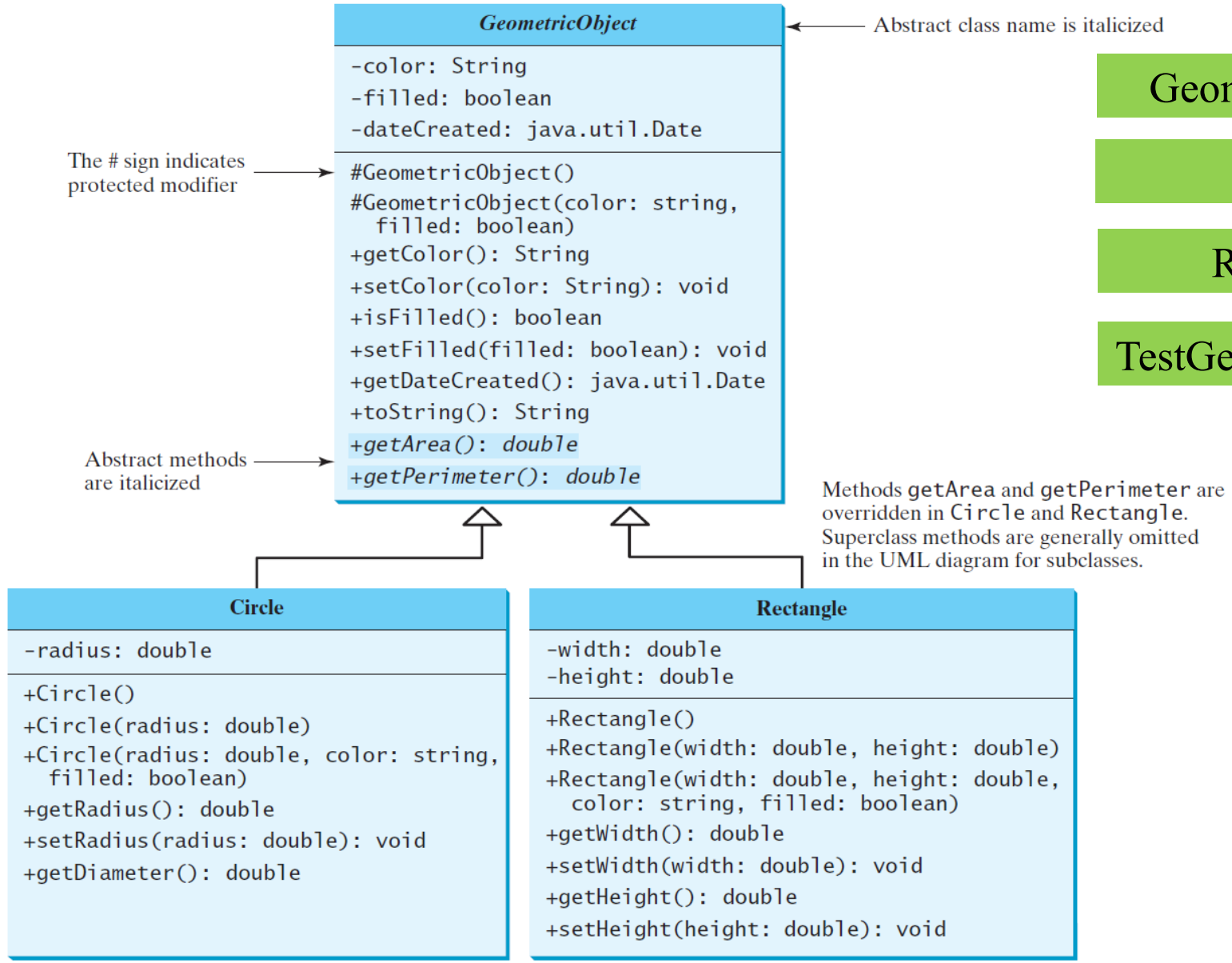
- Such methods are referred to as *abstract methods* and are denoted using the **abstract** modifier in the method header.

- After you define the methods in **GeometricObject**, it becomes an abstract class. Abstract classes are denoted using the **abstract** modifier in the class header.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes and Abstract Methods



GeometricObject

Circle

Rectangle

TestGeometricObject



Copyright © 2022 Pearson Education Ltd. All rights reserved.

abstract method in *abstract* class

- An abstract method cannot be contained in a nonabstract class.
- If a subclass of an abstract superclass does not implement all the abstract methods, the subclass must be defined abstract.
- In other words, in a nonabstract subclass extended from an abstract class, all the abstract methods must be implemented, even if they are not used in the subclass.
- Also note abstract methods are nonstatic.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

object cannot be created from *abstract* class

- An abstract class cannot be instantiated using the *new* operator.
- We can still define its constructors, which are invoked in the constructors of its subclasses.
  - For instance, the constructors of GeometricObject are invoked in the Circle class and the Rectangle class.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

abstract class without abstract method

- A class that contains abstract methods must be abstract.
- However, it is possible to define an abstract class that contains no abstract methods. In this case, you cannot create instances of the class using the new operator.
- This class is used as a base class for defining a new subclass.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

superclass of *abstract* class may be concrete

- A subclass can be abstract even if its superclass is concrete.
- For example, the Object class is concrete, but its subclasses, such as GeometricObject, may be abstract.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

concrete method overridden to be abstract

- A subclass can override a method from its superclass to define it abstract.
- This is rare, but useful when the implementation of the method in the superclass becomes invalid in the subclass.
- In this case, the subclass must be defined abstract.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

abstract class as type

- You cannot create an instance from an abstract class using the new operator, but an abstract class can be used as a data type.

```
GeometricObject c = new Circle(2);
```

- The following statement, which creates an array whose elements are of GeometricObject type, is correct.

```
GeometricObject[] geo = new GeometricObject[10];
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

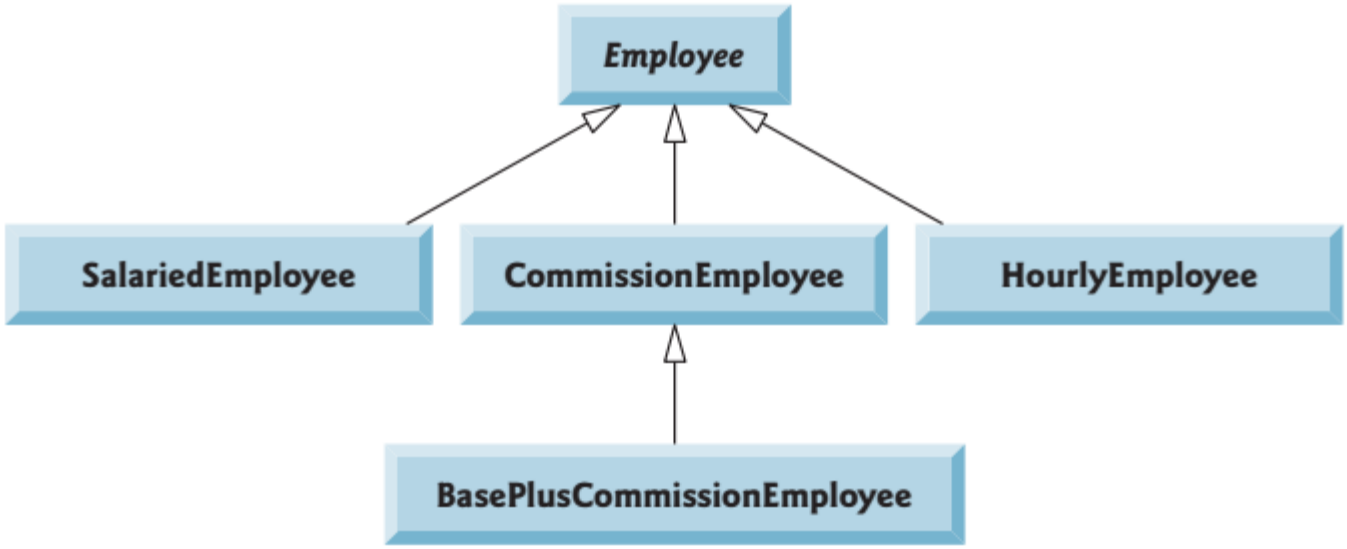
*Case Study: Payroll System Using Polymorphism

- A company pays its employees on a weekly basis.
- The employees are of four types:
  - ***Salaried*** employees are paid a fixed weekly salary regardless of the number of hours worked,
  - ***Hourly*** employees are paid by the hour and receive overtime pay (i.e., 1.5 times their hourly salary rate) for all hours worked in excess of 40 hours,
  - ***Commission*** employees are paid a percentage of their sales and
  - ***Base-salaried commission*** employees receive a base salary plus a percentage of their sales.
- For the current pay period, the company has decided to reward base salaried-commission employees by adding **10%** to their base salaries. The company wants to write an application that performs its payroll calculations polymorphically.

***Java : How To Program** P.J. **Deitel**, H.M. **Deitel**., 9th ed.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

*Case Study: Payroll System Using Polymorphism





***Java : How To Program** P.J. **Deitel**, H.M. **Deitel**., 9th ed.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Superclass *Employee*

- a constructor that takes the first name, last name and social security number as arguments
- get/set methods
- Method toString()
- abstract method *earnings*



Copyright © 2022 Pearson Education Ltd. All rights reserved.

`earnings` and `toString` methods in class hierarchy

	earnings	toString
Employee	abstract	*firstName lastName* social security number: *SSN*
Salaried-Employee	weeklySalary	salaried employee: *firstName lastName* social security number: *SSN* weekly salary: *weeklySalary*

Hourly-Employee	if (hours <= 40)
	  wage * hours
	else if (hours > 40)
	{
	  40 * wage +
	  (hours - 40) *
	  wage * 1.5
	}
		hourly employee: *firstName lastName*
		social security number: *SSN*
		hourly wage: *wage*; hours worked: *hours*

Commission-Employee	commissionRate * grossSales	commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales*; commission rate: *commissionRate*
BasePlus-Commission-Employee	(commissionRate * grossSales) + baseSalary	base salaried commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales*; commission rate: *commissionRate*; base salary: *baseSalary*



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Concrete Subclass SalariedEmployee

- SalariedEmployee extends class Employee
- constructor that takes a first name, a last name, a social security number and a weekly salary as arguments
- get/set methods
- Method toString()
- Method earnings



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Concrete Subclass HourlyEmployee

- HourlyEmployee extends class Employee
- constructor that takes a first name, a last name, a social security number, an hourly wage and the number of hours worked.
- get/set methods
- Method toString()
- Method earnings



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Concrete Subclass CommissionEmployee

- CommissionEmployee extends class Employee
- constructor that takes a first name, a last name, a social security number, a sales amount and a commission rate;
- get/set methods
- Method toString()
- Method earnings



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Indirect Concrete Subclass BasePlusCommissionEmployee

- BasePlusCommissionEmployee extends class CommissionEmployee
- constructor that takes a first name, a last name, a social security number, a sales amount, a commission rate and a base salary.
- get/set methods
- Method toString()
- Method earnings



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Sample Output:

```
Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned: $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned: $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned: $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
earned: $500.00

Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
new base salary with 10% increase is: $330.00
earned $530.00

Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Abstract Classes & Methods Review

- What is an abstract method?
- What is an abstract class?
- Answer the following questions (True/False)?
  1. An abstract class can be instantiated.
  2. An abstract class must have at least one abstract method.
  3. The subclasses of an abstract class must implement all the abstract methods defined in the superclass.
     - What happens if the subclass of an abstract class does not implement all the abstract methods?
  4. A subclass of a concrete class cannot be abstract.
  5. An abstract class cannot be used as a data type.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Interfaces

What is an interface?

Why is an interface useful?

How do you define an interface?

How do you use an interface?



Copyright © 2022 Pearson Education Ltd. All rights reserved.

What is an interface? Why is an interface useful?

- An interface is a class-like construct that contains only *constants* and *abstract methods*.
- In many ways, an interface is similar to an abstract class, but the intent of an interface is to specify common behavior for objects of *related classes* or *unrelated classes*.
- For example, you can specify that the objects are comparable, edible, cloneable using appropriate interfaces.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Define an Interface

To distinguish an interface from a class, Java uses the following syntax to define an interface:

```
public interface InterfaceName {
  constant declarations;
  abstract method signatures;
}
```

Example:

```
public interface Edible {
  /** Describe how to eat */
  public abstract String howToEat();
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Interface is a Special Class

- An interface is treated like a special class in Java.
- Each interface is compiled into a separate bytecode file, just like a regular class.
- Like an abstract class, you cannot create an instance from an interface using the **new** operator.
- You can use an interface as a data type for a variable, as the result of casting, and so on.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example

- You can now use the **Edible** interface to specify whether an object is edible.

- This is accomplished by letting the class for the object implement this interface using the **implements** keyword.

- For example, the classes Chicken and Fruit implement the Edible interface (See TestEdible).

```
class Fruit implements Edible
```



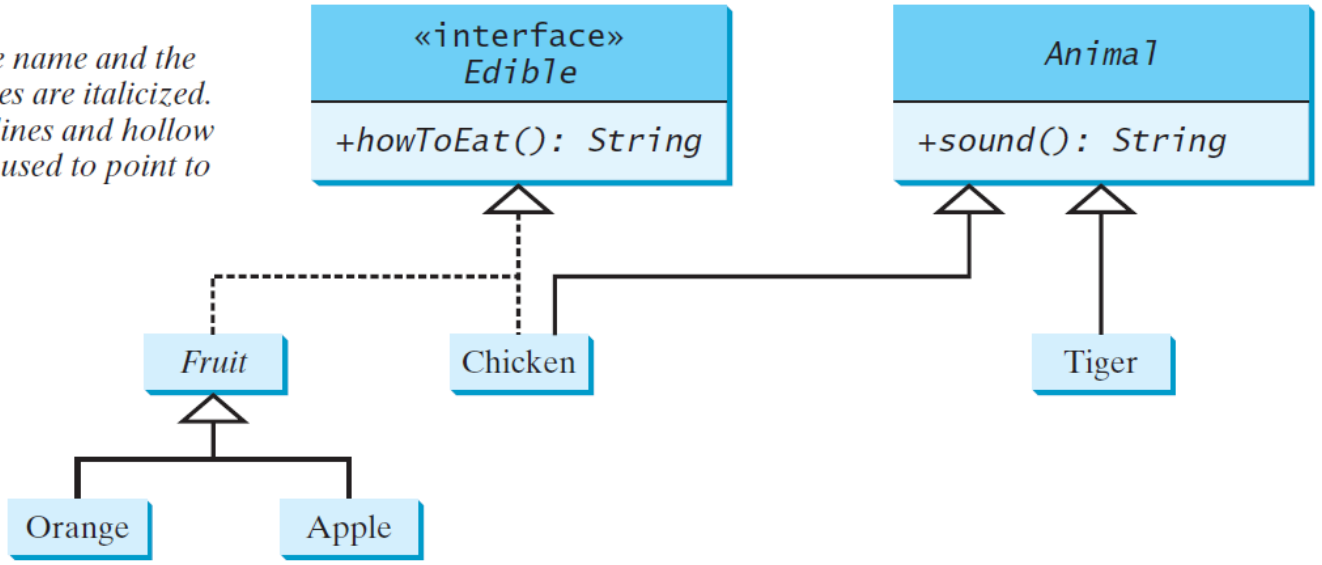
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example

Edible

TestEdible

Notation:
The interface name and the method names are italicized. The dashed lines and hollow triangles are used to point to the interface.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Omitting Modifiers in Interfaces

- All data fields are *public final static* in an interface.
- All methods are *public abstract* in an interface.
- For this reason, these modifiers can be omitted, as shown below:

```
public interface T1 {
  public static final int K = 1;

  public abstract void p();
}
```

Equivalent

```
public interface T1 {
  int K = 1;

  void p();
}
```

A constant defined in an interface can be accessed using syntax InterfaceName.CONSTANT_NAME (e.g., T1.K).

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exercise #1: What is wrong in the following code?

```
interface A {
      void m1();
}
class B implements A {
      void m1() {
            System.out.println("m1");
      }
}
```

attempting to assign weaker access privileges;



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Comparable Interface

The **Comparable** interface defines the **compareTo** method for comparing objects.

```
// This interface is defined in java.lang package
package java.lang;

public interface Comparable<E> {
  public int compareTo(E o);
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The Comparable Interface

```
public interface Comparable<E> {
  public int compareTo(E o);
}
```

- The **Comparable** interface is a generic interface.

- The generic type **E** is replaced by a concrete type when implementing this interface.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The toString, equals, and hashCode Methods

Each wrapper class overrides the toString, equals, and hashCode methods defined in the Object class. Since all the numeric wrapper classes and the Character class implement the Comparable interface, the compareTo method is implemented in these classes.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Integer and BigInteger Classes

```
public class Integer extends Number
    implements Comparable<Integer> {
  // class body omitted

  @Override
  public int compareTo(Integer o) {
    // Implementation omitted
  }
}
```

```
public class BigInteger extends Number
    implements Comparable<BigInteger> {
  // class body omitted

  @Override
  public int compareTo(BigInteger o) {
    // Implementation omitted
  }
}
```

String and Date Classes

```
public class String extends Object
    implements Comparable<String> {
  // class body omitted

  @Override
  public int compareTo(String o) {
    // Implementation omitted
  }
}
```

```
public class Date extends Object
    implements Comparable<Date> {
  // class body omitted

  @Override
  public int compareTo(Date o) {
    // Implementation omitted
  }
}
```


Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example

Use the **compareTo** method to compare two numbers, two strings, and two dates. For example, the following code

```
1  System.out.println(new Integer(3).compareTo(new Integer(5)));
2  System.out.println("ABC".compareTo("ABE"));
3  java.util.Date date1 = new java.util.Date(2013, 1, 1);
4  java.util.Date date2 = new java.util.Date(2012, 1, 1);
5  System.out.println(date1.compareTo(date2));
```

displays

```
-1    // 3 is less then 5
-2    // ABC is less than ABE
 1    // date1 is greater than date2.
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Generic `sort` Method

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object. All the following expressions are **true**.

```
n instanceof Integer
n instanceof Object
n instanceof Comparable
```

```
s instanceof String
s instanceof Object
s instanceof Comparable
```

```
d instanceof java.util.Date
d instanceof Object
d instanceof Comparable
```

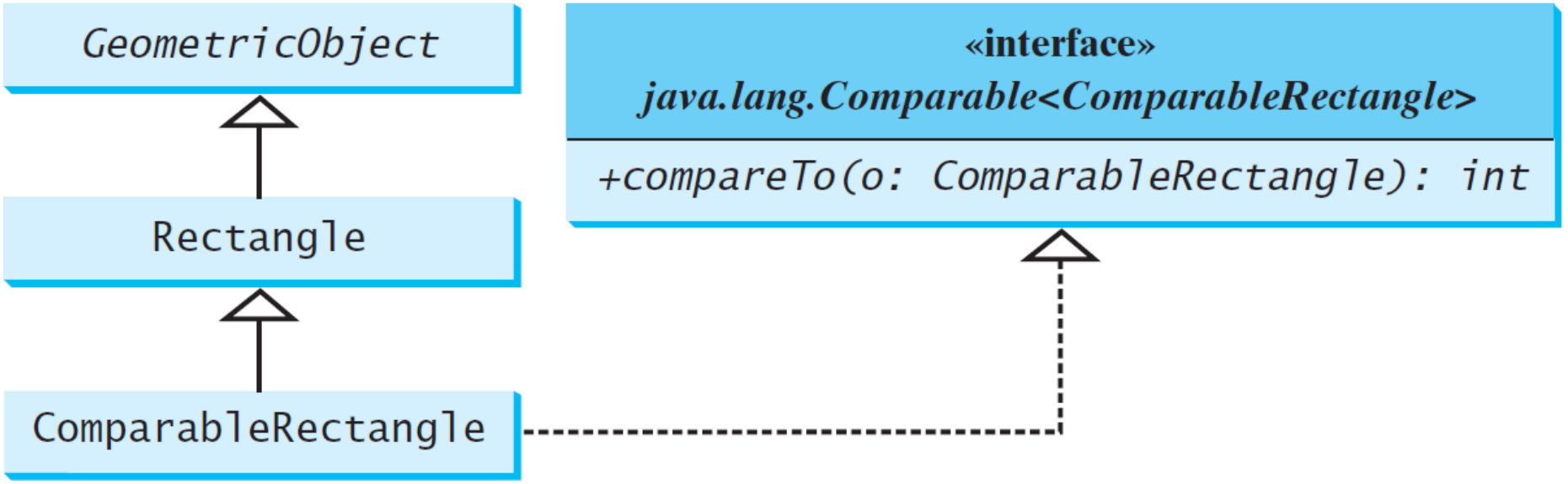
The java.util.Arrays.sort(array) method requires that the elements in an array are instances of Comparable<E>.

SortComparableObjects



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Defining Classes to Implement Comparable



ComparableRectangle

SortRectangles

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exercise #2: What is wrong in the following code?

```
public class Test {
    public static void main(String[] args) {
       Person[] persons = {new Person(3), new Person(4), new
Person(1)};
       java.util.Arrays.sort(persons);
    }
}
class Person {
    private int id;
    Person(int id) {
       this.id = id;
    }
}
```

Exception in thread "main“ java.lang.ClassCastException:
Person cannot be cast to java.lang.Comparable



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The `Cloneable` Interfaces

- *The* **Cloneable** *interface specifies that an object can be cloned.*
- Marker Interface: An empty interface.
- A marker interface does not contain constants or methods. It is used to denote that a class possesses certain desirable properties. A class that implements the Cloneable interface is marked cloneable, and its objects can be cloned using the clone() method defined in the Object class.

```
package java.lang;
public interface Cloneable {
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Examples

```
int[] list1 = {1, 2};
int[] list2 = list1.clone();
list1[0] = 7;
list2[1] = 8;
System.out.println("list1 is " + list1[0] + ", " + list1[1]);
System.out.println("list2 is " + list2[0] + ", " + list2[1]);
```

displays

```
list1 is 7, 2
list2 is 1, 8
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Implementing Cloneable Interface

- To define a custom class that implements the **Cloneable** interface, the class must override the **clone()** method in the **Object** class.

- If we try to create a clone of an object instance of a class that does not implement the **Cloneable** interface, it throws **CloneNotSupportedException.**



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Implementing Cloneable Interface

- We can override the **clone()** method from the **Object** class to create custom clones.
  - The **clone** method in the **Object** class creates a new instance of the class of this object and initializes all its fields with exactly the contents of the corresponding fields of this object, as if by assignment; the contents of the fields are not themselves cloned.
  - The **clone** method in the **Object** class has a **protected** modifier. To use this method in any package, the implementing class should change the visibility modifier to **public**.
- The **clone()** method returns an **Object** that needs to be casted.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Implementing Cloneable Interface

The following code defines a class named House that implements Cloneable and Comparable.

House



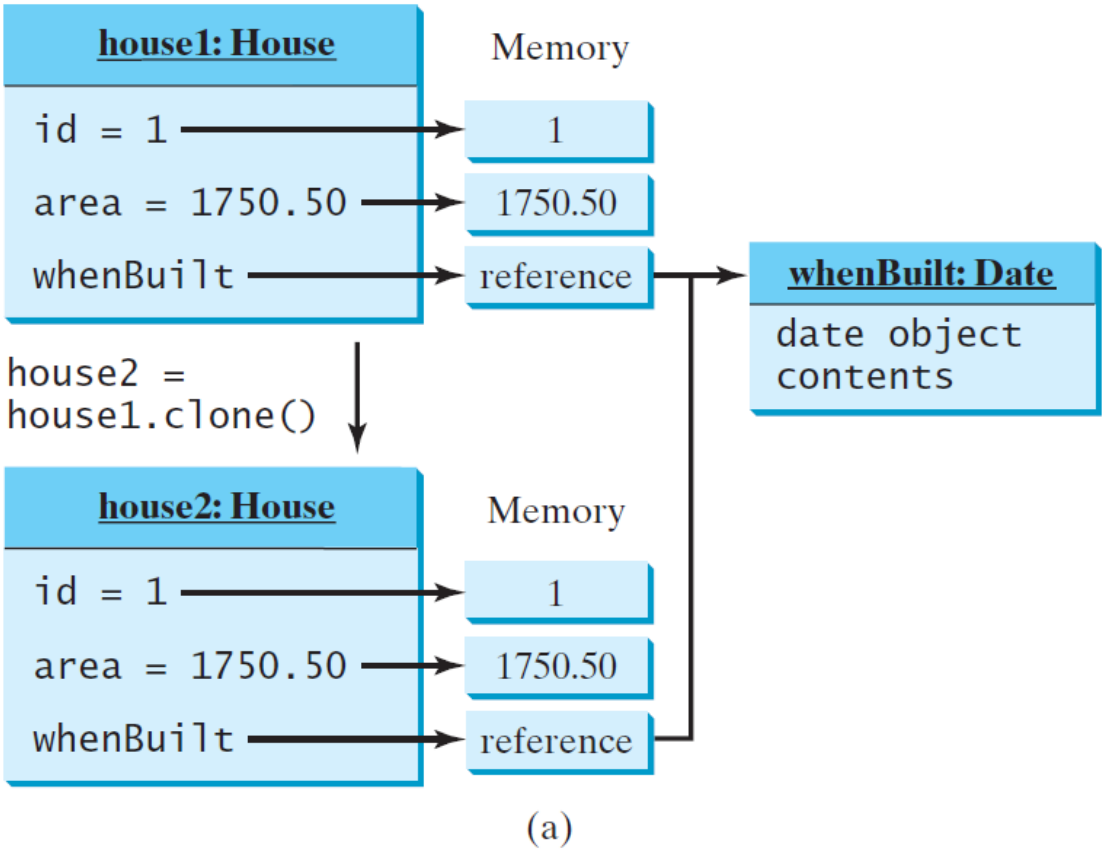
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Shallow vs. Deep Copy

House house1 = new House(1, 1750.50);

House house2 = (House)house1.clone();

Shallow Copy



(a)

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Deep Copy

To perform a deep copy for a **House** object, replace the **clone()** method in lines 26–28 with the following code:

```
public Object clone() {
  try {
        // Perform a shallow copy
        House houseClone = (House)super.clone();
        // Deep copy on whenBuilt
        houseClone.whenBuilt = (java.util.Date)(whenBuilt.clone());
        return houseClone;
  }
  catch (CloneNotSupportedException ex) {
        return null;
  }
}
```



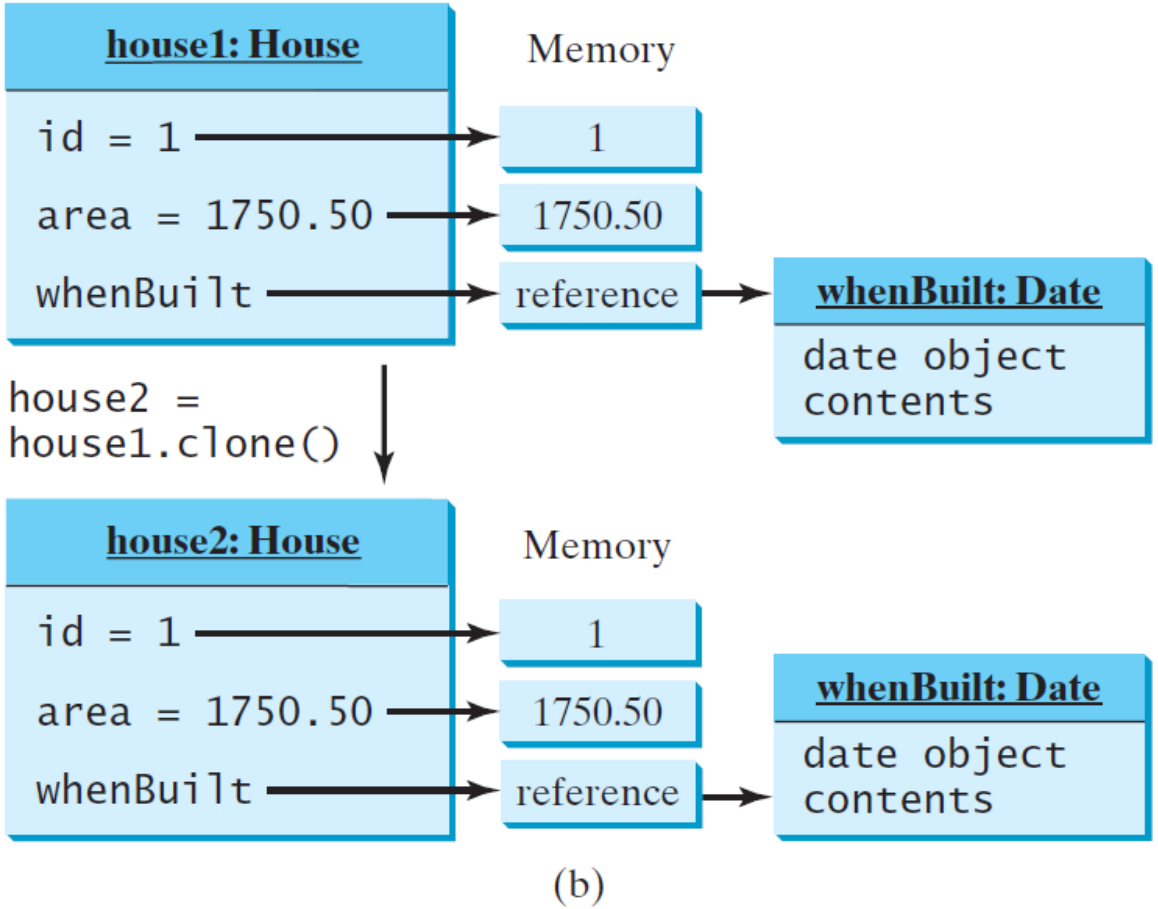
Copyright © 2022 Pearson Education Ltd. All rights reserved.

Shallow vs. Deep Copy

House house1 = new House(1, 1750.50);

House house2 = (House)house1.clone();

Deep Copy



(b)

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exercise #3: Show the output of the following code:

```
java.util.Date date = new java.util.Date();

java.util.Date date1 = date;

java.util.Date date2 = (java.util.Date)(date.clone());

System.out.println(date == date1);

System.out.println(date == date2);

System.out.println(date.equals(date2));
```

True

False

True



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Interfaces vs. Abstract Classes

In an interface, the data must be constants; an abstract class can have all types of data.

Each method in an interface has only a signature without implementation; an abstract class can have concrete methods.

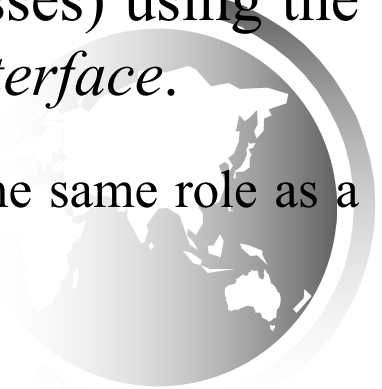
	Variables	*Constructors*	*Methods*
Abstract class	No restrictions.	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator.	No restrictions.
Interface	All variables must be `public static final`.	No constructors. An interface cannot be instantiated using the new operator.	All methods must be public abstract instance methods



Copyright © 2022 Pearson Education Ltd. All rights reserved.

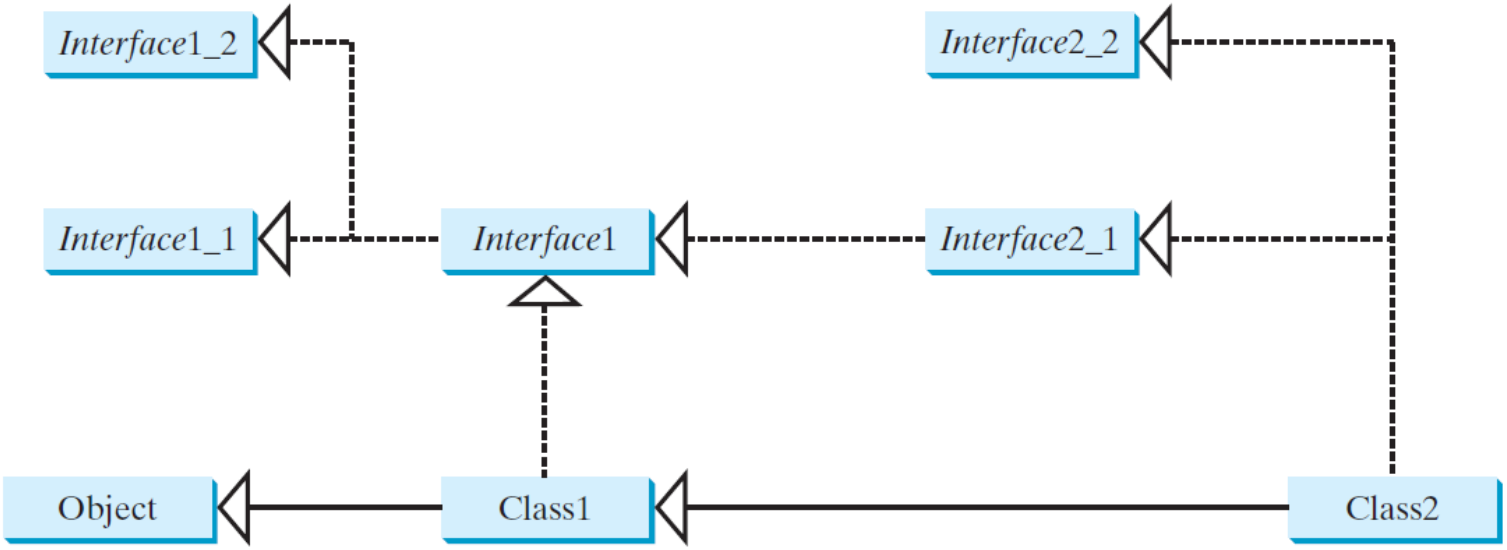
Interfaces vs. Abstract Classes, cont.

- All classes share a single root, the **Object** class, but there is no single root for interfaces.
- Like a class, an interface also defines a *type*.
  - A variable of an interface type can reference any instance of the class that implements the interface.
  - You can use an interface as a data type and cast a variable of an interface type to its subclass, and vice versa.
- An interface can inherit other interfaces (but not classes) using the **extends** keyword. Such an interface is called a *subinterface*.
  - If a class implements an interface, this interface plays the same role as a superclass.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Interfaces vs. Abstract Classes, cont.



Suppose that **c** is an instance of **Class2**.

c is also an instance of **Object**, **Class1**, **Interface1**, **Interface1_1**, **Interface1_2**, **Interface2_1**, and **Interface2_2**.

A class implementing **Interface2_1** must implement the abstract methods defined in **Interface1**, **Interface1_1,** and **Interface1_2**.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Caution: conflicting interfaces

- Errors detected by the compiler:
  - If a class implements two interfaces with conflict information:
    - two same constants with different values,

      or

    - two methods with same signature but different return type



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Whether to use an interface or a class?

- Abstract classes and interfaces can both be used to model common features.

- How do you decide whether to use an interface or a class?

- In general, a strong is-a relationship that clearly describes a parent-child relationship should be modeled using classes.
  - For example, a staff member is a person.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Whether to use an interface or a class?

- A weak is-a relationship, also known as an is-kind-of relationship, indicates that an object possesses a certain property.

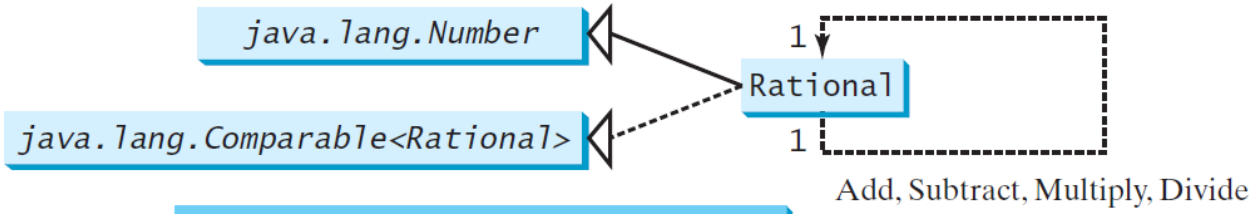
- A weak is-a relationship can be modeled using interfaces.
  - For example, all strings are comparable, so the **String** class implements the **Comparable** interface.

- You can also use interfaces to circumvent single inheritance restriction if multiple inheritance is desired.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The `Rational` Class



Rational
-numerator: long -denominator: long
+Rational() +Rational(numerator: long, denominator: long) +getNumerator(): long +getDenominator(): long +add(secondRational: Rational): Rational +subtract(secondRational: Rational): Rational +multiply(secondRational: Rational): Rational +divide(secondRational: Rational): Rational +toString(): String -gcd(n: long, d: long): long

The numerator of this rational number.
The denominator of this rational number.
Creates a rational number with numerator 0 and denominator 1.
Creates a rational number with a specified numerator and denominator.
Returns the numerator of this rational number.
Returns the denominator of this rational number.
Returns the addition of this rational number with another.
Returns the subtraction of this rational number with another.
Returns the multiplication of this rational number with another.
Returns the division of this rational number with another.
Returns a string in the form "numerator/denominator." Returns the numerator if denominator is 1.
Returns the greatest common divisor of n and d.

Rational TestRationalClass

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Class Design Guidelines



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Coherence

- A class should describe a **single entity**, and all the class operations should logically fit together to support a coherent purpose.
- You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Coherence, cont.

- A single entity with too many responsibilities can be broken into several classes to separate responsibilities.
- The classes String, StringBuilder, and StringBuffer all deal with strings, for example, but have different responsibilities.
- The String class deals with immutable strings, the StringBuilder class is for creating mutable strings, and the StringBuffer class is similar to StringBuilder except that StringBuffer contains synchronized methods for updating strings.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Consistency

- Follow standard Java programming style and naming conventions.
- Choose informative names for classes, data fields, and methods.
- A popular style is to place the data declaration before the constructor and place constructors before methods.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Consistency, cont.

- Always provide a constructor and initialize variables to avoid programming errors.
- Provide a public no-arg constructor and override the equals method and the toString method defined in the Object class whenever possible.
- In general, you should consistently provide a public no-arg constructor for constructing a default instance. If a class does not support a no-arg constructor, document the reason.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reuse

- Classes are designed for reuse.
- Users can incorporate classes in many different combinations, orders, and environments.
- Therefore, you should design a class that imposes no restrictions on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Encapsulation

- A class should use the **private** modifier to hide its data from direct access by clients. This makes the class easy to maintain.
- Provide a getter method only if you want the data field to be readable, and provide a setter method only if you want the data field to be updateable.
- A class should also hide methods not intended for client use. The **gcd** method in the **Rational** class is private, for example, because it is only for internal use within the class.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Using Visibility Modifiers

- Each class can present two contracts – one for the *users* of the class and one for the *extenders* of the class.
- Make the fields private and accessor methods public if they are intended for the users of the class.
- Make the fields or method protected if they are intended for extenders of the class.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Using the static Modifier

A property that is shared by all the instances of the class should be declared as a static property.



Copyright © 2022 Pearson Education Ltd. All rights reserved.